

Chapitre 4 : Exceptions et gestion d'erreurs

I Introduction

Une exception permet d'interrompre le flux d'exécution normal d'un programme. Elle survient généralement lors d'une mauvaise interaction avec l'environnement extérieur ou lorsqu'une fonction est appliquée à des valeurs non attendues (par exemple, une division par zéro).

✗ **Attention** ✗ Il ne faut absolument pas confondre les exceptions avec les erreurs de typage.

- Les **erreurs de typage** sont détectées à la **compilation** (statiquement).
- Les **exceptions** se produisent à l'**exécution** du programme (dynamiquement).

Le recours aux exceptions est toujours préférable au renvoi de valeurs légales mais artificielles (comme renvoyer -1 pour le minimum d'une liste vide d'entiers, ce qui fausserait les calculs si la liste contenait réellement -1). Si une exception n'est pas rattrapée, elle provoque l'arrêt immédiat du programme.

II Exceptions prédéfinies de OCaml

La bibliothèque standard de OCaml fournit plusieurs exceptions courantes prêtes à l'emploi :

- `Division_by_zero` : Déclenchée lors d'une division entière par 0.
- `Failure` : Déclenchée par la fonction `failwith "message"`.
- `Invalid_argument` : Indique qu'un argument ne respecte pas les préconditions (ex: index hors des bornes).
- `Not_found` : Déclenchée lorsqu'un élément cherché n'existe pas.

💡 **Exemple** : Exemples d'expressions provoquant des exceptions natives :

```
1 1 / 0;; (* Exception: Division_by_zero *)
2 List.hd [];; (* Exception: Failure "hd" *)
3 String.get "hello" 17; (* Exception: Invalid_argument "index out of bounds" *)
4 List.assoc "z" [("a", 5)];; (* Exception: Not_found *)
```

A La fonction `failwith`

La fonction prédéfinie `failwith : string -> 'a` est extrêmement utile car son type de retour est polymorphe (`'a`), ce qui lui permet de s'insérer dans n'importe quelle branche d'un filtrage ou d'une condition sans provoquer de conflit de type.

💡 **Exemple** : Utilisation de `failwith` pour traiter le cas d'une liste vide :

```
1 let rec minlist l =
2   match l with
3   | [] -> failwith "Une liste vide n'a pas de minimum"
4   | [x] -> x
5   | h :: r -> min h (minlist r);
```

III Déclaration et levée d'exceptions personnalisées

Il est possible de définir ses propres exceptions à l'aide du mot-clé `exception`. Elles peuvent être de simples constantes ou emporter des valeurs (arguments). Pour déclencher manuellement une exception, on utilise le mot-clé `raise`.

Définition : Une exception se déclare de la manière suivante :

```
exception Mon_Exception OU exception Mon_Exception of type
```

 **Exemple :** Déclaration et levée d'exceptions avec ou sans paramètres :

```
1 exception Empty;
2 exception Bug of string * int;
3
4 let verif x =
5   if x < 0 then raise (Bug ("valeur negative", x)) else x;
```

IV Rattrapage d'exceptions

Rattraper (ou intercepter) une exception permet d'éviter l'arrêt du programme en proposant une solution de secours ou un comportement alternatif.

A La structure `try ... with`

La syntaxe pour intercepter une exception repose sur un filtrage par motif dédié à l'aide de la structure `try ... with`.

Méthode : Structure du `try ... with`

On place l'expression principale immédiatement après le mot-clé `try`, puis on énumère les différents motifs d'exceptions à intercepter après le mot-clé `with`.

```
1 try expression_principale with
2 | Exception_1 -> expression_secours_1
3 | Exception_2 -> expression_secours_2
```

Si l'expression_principale s'évalue sans encombre, son résultat est renvoyé. Si elle lève une exception, celle-ci est comparée aux motifs après le `with`.

 **Exemple :** Exemple de fonction de division sécurisée renvoyant une valeur par défaut :

```
1 let div_safe x y =
2   try x / y with
3   | Division_by_zero -> 0;;
```

B Filtrage combiné avec `match ... with exception`

Il est également possible d'intercepter les exceptions directement au sein d'un bloc `match ... with standard` en faisant précéder le motif du mot-clé `exception`.

 **Exemple :** Exemple de filtrage combinant valeurs de retour et exceptions :

```
1 let g x =
2   match f x with
3   | 'd' -> "Lettre d"
```

```
4 | 'e' -> "Lettre e"
5 | exception (Bug (msg, _)) -> "Une erreur est survenue : " ^ msg
6 | _ -> "Autre lettre";;
```

V Assertions

Une assertion permet de vérifier la validité d'un invariant ou d'une précondition au cours de l'exécution d'un programme.

Définition : Le mot-clé `assert` est suivi d'une expression booléenne. Si l'expression s'évalue à `false`, OCaml lève automatiquement l'exception `Assert_failure`, qui indique le nom du fichier, le numéro de la ligne et la colonne de l'erreur.

❗ Remarque : Les assertions sont particulièrement utiles lors des phases de test ou de débogage. Si nécessaire, on peut désactiver la vérification des assertions lors de la compilation du programme afin d'optimiser les performances en utilisant l'option `-noassert` (ex : `ocamlc -noassert programme.ml`).

💡 **Exemple :** Utilisation d'un assert pour garantir qu'un indice de tableau est positif :

```
1 let acces_table t i =
2   assert (i >= 0);
3   t.(i);;
```

 **Application :** Écrire une fonction `assoc_safe` : `'a -> ('a * 'b) list -> 'b -> 'b` qui cherche une clé dans une liste d'association à l'aide de `List.assoc`. Si la clé n'est pas trouvée, la fonction devra intercepter l'exception `Not_found` pour renvoyer une valeur par défaut passée en argument.

[illegible]

Contents

I	Introduction	1
II	Exceptions prédéfinies de OCaml	1
A	La fonction <code>failwith</code>	1
III	Déclaration et levée d'exceptions personnalisées	2
IV	Rattrapage d'exceptions	2
A	La structure <code>try ... with</code>	2
B	Filtrage combiné avec <code>match ... with exception</code>	2
V	Assertions	3